

Focal Loss for Imbalance, Worked Out by Hand

Down-weighting easy examples so the rare class gets heard

A complete numerical walk-through of one loss function: $\text{FL}(p_t) = -\alpha(1 - p_t)^\gamma \log p_t$, where p_t is the model's predicted probability of the *true* class. We start from plain cross-entropy $\text{CE} = -\log p_t$, attach the *modulating factor* $(1 - p_t)^\gamma$, and compute every cell of a CE-vs-FL table by hand. Nothing is hand-waved — every number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It takes four probabilities the model might assign to the correct class — $p_t \in \{0.9, 0.7, 0.5, 0.1\}$, spanning easy to very hard — and evaluates both losses at $\gamma \in \{0, 1, 2\}$. With $\gamma = 0$ focal loss *is* cross-entropy, so the table contains its own control column. We then bring in α and show how it re-balances a class that appears only 3 times in a batch of 32.

How to read this. Each mechanism gets a worked block with exact numbers and a coloured box isolating the idea. The headline fact — that at $\gamma = 2$ an easy example ($p_t = 0.9$) has its loss cut $\sim 100\times$ while a hard one ($p_t = 0.5$) is cut only $4\times$ — is derived twice: once cell-by-cell, once as a batch-level *gradient-share* computation that shows the rare class going from 6% to 62% of the signal.

Concepts covered, in order: cross-entropy as $-\log p_t$ · the modulating factor $(1 - p_t)^\gamma$ · the full CE-vs-FL table at three γ 's · the $100\times$ vs $4\times$ asymmetry · the batch gradient-share argument · the α class-balancing term · how γ and α are tuned together · why focal loss complements, not replaces, weighted sampling.

1 The setup: one probability, two losses

A classifier outputs logits; a softmax turns them into class probabilities. For a single training example, the only number the loss cares about is p_t , the probability the model assigned to the *correct* label. Everything else is bookkeeping. Plain cross-entropy on that example is

$$\text{CE}(p_t) = -\log p_t.$$

When the model is confident and right (p_t near 1) the loss is tiny; when it is confident and wrong (p_t near 0) the loss explodes. That is correct behaviour — but it treats a barely-needed nudge on an easy example and an urgent correction on a hard one as differences only of *degree*. Under heavy class imbalance, the sheer *count* of easy examples means their small individual losses still sum to dominate the gradient.

Focal loss inserts a multiplicative gate in front of CE:

$$\text{FL}(p_t) = -\alpha(1-p_t)^\gamma \log p_t = \alpha(1-p_t)^\gamma \text{CE}(p_t)$$

The factor $(1-p_t)^\gamma$ is the **modulating factor**; α is a per-class weight. We will fix $\alpha = 1$ for Sections 2–4 to isolate γ , then restore α in Section 5.

Quantity	Symbol	Role
True-class probability	p_t	softmax output for the correct label
Cross-entropy	$\text{CE} = -\log p_t$	baseline per-example loss
Focusing parameter	$\gamma \geq 0$	how hard to down-weight easy examples
Modulating factor	$(1-p_t)^\gamma$	shrinks loss when p_t is high
Class weight	α	re-balances rare vs. common classes
Focal loss	$\text{FL} = \alpha(1-p_t)^\gamma \text{CE}$	the gated loss

Intuition

Reshape the loss curve, do not replace it. Cross-entropy already says ”hard examples matter more”; focal loss says it *louder* by multiplying each example’s loss by $(1-p_t)^\gamma$, a number that is near 0 for the many easy/majority examples and near 1 for the few hard/rare ones. The thousands of confidently-correct background examples stop drowning out the handful of cases the model actually needs to learn.

2 Step 1 — The modulating factor $(1-p_t)^\gamma$

Set $\alpha = 1$. The whole behaviour of focal loss lives in the factor $(1-p_t)^\gamma$. Read it as a knob on a dimmer switch attached to each example:

- $\gamma = 0$: the factor is $(1-p_t)^0 = 1$ for *every* p_t . The dimmer is fully open — $\text{FL} = \text{CE}$ exactly. This is the control case.
- $\gamma > 0$: the factor is < 1 whenever $p_t > 0$, and it shrinks *faster* the closer p_t is to 1. Easy examples get dimmed hard; hard examples barely at all.

Evaluate the factor at the two anchor cases from the course, both at $\gamma = 2$:

$$\underbrace{(1 - 0.9)^2 = (0.1)^2 = 0.010}_{\text{easy: } p_t=0.9}, \quad \underbrace{(1 - 0.5)^2 = (0.5)^2 = 0.250}_{\text{hard: } p_t=0.5}.$$

The easy example's loss is multiplied by 0.010 — a cut of $1/0.010 = 100\times$. The hard example's loss is multiplied by 0.250 — a cut of only $1/0.250 = 4\times$. Same γ , wildly different treatment, and *that asymmetry is the entire point*.

Mini-example — this operation in isolation

Why the cut is so uneven. The factor is a power of the model's *error* $(1 - p_t)$. For the easy example the error is 0.1; squaring a small number makes it tiny (0.01). For the hard example the error is 0.5; squaring it barely moves it (0.25). Raising γ from 2 to 3 would cut the easy case to $(0.1)^3 = 0.001$ (1000 $\times$) but the hard case only to $(0.5)^3 = 0.125$ (8 $\times$). The gap between "easy" and "hard" widens with γ — which is exactly the lever you want, and exactly the lever that can be pushed too far.

3 Step 2 — The full CE-vs-FL table (exact, $\alpha = 1$)

Now compute every cell. For each p_t we first get $\text{CE} = -\log p_t$, then multiply by $(1 - p_t)^\gamma$ for $\gamma = 0, 1, 2$. The $\gamma = 0$ column reproduces CE, confirming the formula reduces correctly.

p_t	$\text{CE} = -\log p_t$	$(1-p_t)^1$	$\text{FL}_{\gamma=1}$	$(1-p_t)^2$	$\text{FL}_{\gamma=2}$
0.9	0.105	0.100	0.011	0.010	0.001
0.7	0.357	0.300	0.107	0.090	0.032
0.5	0.693	0.500	0.347	0.250	0.173
0.1	2.303	0.900	2.072	0.810	1.865

Two columns are worth reading vertically. Sample worked cells (rounded to 3 dp, computed in float64):

$$p_t=0.9, \gamma=2 : \text{FL} = 0.010 \times 0.105 = 0.001 \quad (\text{was } 0.105; \div 100)$$

$$p_t=0.5, \gamma=2 : \text{FL} = 0.250 \times 0.693 = 0.173 \quad (\text{was } 0.693; \div 4)$$

$$p_t=0.1, \gamma=2 : \text{FL} = 0.810 \times 2.303 = 1.865 \quad (\text{was } 2.303; \div 1.23)$$

The very hard example ($p_t = 0.1$) keeps $1.865/2.303 = 81\%$ of its loss — focal loss hardly touches it. The easy example keeps 1%. The ratio of "attention" the model pays to the hard case over the easy case jumped from $2.303/0.105 \approx 21.9$ under CE to $1.865/0.001 \approx 1770$ under $\text{FL}_{\gamma=2}$.

Watch out

$\gamma = 0$ recovers cross-entropy *exactly* — it is not an approximation. If you ever set γ to 0 "to be safe," you have simply turned focal loss off. The down-weighting only exists for $\gamma > 0$; the typical value is $\gamma \approx 2$.

4 Step 3 — Why a 100× cut matters: the gradient-share argument

A per-example cut is interesting; the batch effect is decisive. Imbalance means the *counts* are lopsided, so what governs learning is the *total* loss each group contributes. Take a deliberately extreme batch: 100 easy background examples all at $p_t = 0.9$, and 1 hard rare-class example at $p_t = 0.5$.

Under cross-entropy. Each group’s total loss is its count times its per-example CE:

$$\begin{aligned} \text{easy: } 100 \times (-\log 0.9) &= 100 \times 0.105 = 10.536, \\ \text{hard: } 1 \times (-\log 0.5) &= 1 \times 0.693 = 0.693. \end{aligned}$$

The rare example’s share of the total signal is

$$\frac{0.693}{10.536 + 0.693} = \frac{0.693}{11.229} = 0.0617 = \boxed{6.17\%}.$$

Ninety-four percent of the gradient is spent re-confirming things the model already gets right.

Under focal loss ($\gamma = 2$, $\alpha = 1$). Multiply each per-example loss by its modulating factor first:

$$\begin{aligned} \text{easy: } 100 \times 0.010 \times 0.105 &= 100 \times 0.00105 = 0.105, \\ \text{hard: } 1 \times 0.250 \times 0.693 &= 1 \times 0.17329 = 0.173. \end{aligned}$$

Now the rare example’s share is

$$\frac{0.173}{0.105 + 0.173} = \frac{0.173}{0.279} = 0.6219 = \boxed{62.19\%}.$$

The rare class went from 6.17% to 62.19% of the learning signal — a $62.19/6.17 \approx 10.07\times$ boost in relative attention — purely from the modulating factor, with no change to the data or sampling.

When this is the right choice

This is the regime focal loss was built for: **extreme** foreground/background imbalance. Dense object detection (the original use case — one image, thousands of trivial background anchor boxes, a few real objects), and anomaly / fraud / AIOps rare-event detection (millions of normal events, a handful of incidents). When the “easy negatives” outnumber the signal 100:1 or worse, plain CE lets them set the agenda; focal loss takes the microphone back.

5 Step 4 — The α term: balancing class frequencies

γ down-weights by *difficulty* (p_t). α down-weights by *class membership* — a fixed per-class scalar applied regardless of p_t . The two are orthogonal knobs, and the course uses both ($\alpha = 0.25$, $\gamma = 2$ by default). Consider the course’s imbalanced batch: 3 positives out of 32, so 29 negatives.

$$\text{freq}_{\text{pos}} = \frac{3}{32} = 0.094, \quad \text{freq}_{\text{neg}} = \frac{29}{32} = 0.906.$$

A common recipe sets each class weight to the inverse of its frequency (normalised so the two halves balance), $\alpha_c = N/(2n_c)$:

$$\alpha_{\text{pos}} = \frac{32}{2 \times 3} = 5.333, \quad \alpha_{\text{neg}} = \frac{32}{2 \times 29} = 0.552.$$

The rare positive’s loss is scaled up $\sim 9.7\times$ relative to a negative (5.333/0.552). Equivalently, the literature’s convention writes α as a single number in $(0, 1)$ given to the rare class (and $1 - \alpha$ to the common one); the $\alpha = 0.25$ default puts weight 0.25 on the rare positives and 0.75 on the abundant negatives, the opposite of naive inverse frequency because γ is *already* doing most of the imbalance work — so α only needs a gentle nudge.

Worked single examples, combining both terms at $\gamma = 2$:

$$\begin{aligned} p_t=0.5, \alpha=0.75 &: 0.75 \times 0.250 \times 0.693 = 0.130, \\ p_t=0.5, \alpha=0.25 &: 0.25 \times 0.250 \times 0.693 = 0.043, \\ p_t=0.9, \alpha=0.25 &: 0.25 \times 0.010 \times 0.105 = 0.000263. \end{aligned}$$

Watch out

Tune γ and α *together*, not in isolation. They both push loss toward the rare/hard cases, so stacking aggressive values double-counts. A too-high γ *starves learning*: if $\gamma = 5$, even a moderately hard example at $p_t = 0.5$ keeps only $(0.5)^5 = 0.031$ of its loss, so almost nothing trains and convergence stalls. Start at $\gamma \approx 2$, then adjust α . And remember: focal loss *complements*, it does not replace, weighted sampling — the two attack imbalance from different ends (loss magnitude vs. how often a class is even drawn into the batch).

6 Focal loss at a glance

#	Quantity	Value here	Computed as
1	CE at $p_t=0.9$	0.105	$-\log 0.9$
2	CE at $p_t=0.5$	0.693	$-\log 0.5$
3	mod. factor, easy, $\gamma=2$	0.010	$(1 - 0.9)^2$
4	mod. factor, hard, $\gamma=2$	0.250	$(1 - 0.5)^2$
5	easy-example cut	100×	$1/0.010$
6	hard-example cut	4×	$1/0.250$
7	FL easy, $\gamma=2, \alpha=1$	0.001	0.010×0.105
8	FL hard, $\gamma=2, \alpha=1$	0.173	0.250×0.693
9	rare-class share, CE	6.17%	$0.693/11.229$
10	rare-class share, FL	62.19%	$0.173/0.279$

Rows 9–10 are the headline: the same batch, the same model outputs, but focal loss hands 10× more of the gradient to the class that matters.

7 Equation cheat-sheet

Cross-entropy (one example)	$\text{CE}(p_t) = -\log p_t$
Modulating factor	$(1 - p_t)^\gamma$ (equals 1 when $\gamma=0$)
Focal loss	$\text{FL}(p_t) = -\alpha (1 - p_t)^\gamma \log p_t$
Down-weight factor	$\text{CE}/\text{FL} = 1/[\alpha(1 - p_t)^\gamma]$
$\gamma=0$ special case	$\text{FL} = \text{CE}$ (focal loss off)
Inverse-freq class weight	$\alpha_c = N/(2 n_c)$
Group total loss	$n_{\text{group}} \times \text{per-example loss}$
Rare-class signal share	$L_{\text{rare}} / (L_{\text{rare}} + L_{\text{easy}})$
Recovering p_t from CE	$p_t = e^{-\text{CE}}$

Appendix A Reference implementation

Every number above is reproducible from the following program. The NumPy block builds the CE-vs-FL table and the gradient-share argument; the PyTorch block is the course's `FocalLoss` module, which gets p_t back from the per-example cross-entropy via $p_t = e^{-\text{CE}}$.

```
import numpy as np

pts = np.array([0.9, 0.7, 0.5, 0.1])
gammas = [0, 1, 2]

# --- CE vs FL table (alpha = 1) ---
ce = -np.log(pts) # [0.105, 0.357, 0.693, 2.303]
for g in gammas:
    mod = (1 - pts) ** g # modulating factor
    fl = mod * ce # focal loss, alpha = 1
    print(f"gamma={g}: mod={np.round(mod,3)} FL={np.round(fl,3)}")
# gamma=0: FL == CE (modulating factor all ones)

# --- the 100x vs 4x asymmetry at gamma = 2 ---
print("easy p=0.9: (0.1)^2 =", (0.1)**2, "-> cut", 1/(0.1)**2, "x") # 0.01 -> 100x
print("hard p=0.5: (0.5)^2 =", (0.5)**2, "-> cut", 1/(0.5)**2, "x") # 0.25 -> 4x

# --- gradient share: 100 easy (p=0.9) + 1 hard (p=0.5) ---
n_easy, p_easy = 100, 0.9
n_hard, p_hard = 1, 0.5
ce_easy, ce_hard = -np.log(p_easy), -np.log(p_hard)

# cross-entropy: hard class share of total loss
share_ce = (n_hard*ce_hard) / (n_easy*ce_easy + n_hard*ce_hard)
print("CE rare-class share = %.4f" % share_ce) # 0.0617

# focal (gamma=2, alpha=1)
fl_easy = (1-p_easy)**2 * ce_easy
fl_hard = (1-p_hard)**2 * ce_hard
share_fl = (n_hard*fl_hard) / (n_easy*fl_easy + n_hard*fl_hard)
print("FL rare-class share = %.4f" % share_fl) # 0.6219
print("boost = %.2fx" % (share_fl/share_ce)) # 10.07x

# --- alpha from class frequencies (3 pos / 32) ---
N, n_pos, n_neg = 32, 3, 29
print("alpha_pos = N/(2*n_pos) =", N/(2*n_pos)) # 5.333
print("alpha_neg = N/(2*n_neg) =", round(N/(2*n_neg),3)) # 0.552

# =====
# PyTorch: the course's FocalLoss module
import torch
import torch.nn as nn

class FocalLoss(nn.Module):
    """FL(p_t) = -alpha * (1 - p_t)^gamma * log(p_t)."""
    def __init__(self, alpha=0.25, gamma=2.0, reduction='mean'):
        super().__init__()
        self.alpha, self.gamma, self.reduction = alpha, gamma, reduction

    def forward(self, logits, targets):
        ce_loss = nn.functional.cross_entropy(logits, targets, reduction='none')
        pt = torch.exp(-ce_loss) # p_t = prob of correct class
```

```
        focal = self.alpha * (1 - pt) ** self.gamma * ce_loss
        if self.reduction == 'mean': return focal.mean()
        if self.reduction == 'sum':  return focal.sum()
        return focal

# imbalanced batch: 3 positives out of 32
logits = torch.randn(32, 2)
targets = torch.zeros(32, dtype=torch.long)
targets[:3] = 1
print("CE   ", nn.CrossEntropyLoss()(logits, targets).item())
print("Focal", FocalLoss(gamma=2.0)(logits, targets).item())
```

— end of document —